

M15 / STUDY COMPANION

M15 Quick Revision Book

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

M15 | QUICK REVISION BOOK 3

DE15-01 | Cloud fundamentals for data engineers 3

DE15-02 | Identity, IAM and least privilege 3

DE15-03 | Azure Storage concepts and OneLake 3

DE15-04 | Fabric workspace and item model 3

DE15-05 | OneLake paths, files, tables and shortcuts 4

DE15-06 | Fabric Lakehouse and Delta tables 4

DE15-07 | Fabric notebooks 4

DE15-08 | Spark in Fabric 4

DE15-09 | Fabric Data Pipelines 5

DE15-10 | SQL analytics endpoint and warehouse choice 5

DE15-11 | Medallion architecture in Fabric 5

DE15-12 | Incremental loads and MERGE in cloud 5

DE15-13 | Permissions, connections and secrets 6

DE15-14 | Monitoring, reliability and cost awareness 6

DE15-15 | Git, deployment and environment separation 6

DE15-16 | End-to-end Fabric data engineering case 6

Read this first

M15 | QUICK REVISION BOOK

Use for retrieval after normal study. This is not the learner book and should not be your first exposure.

DE15-01 | Cloud fundamentals for data engineers

Core: Cloud DE = managed platform plus your explicit responsibilities for data, identity, orchestration, evidence and cost. Diagnose by boundary, not by brand name.

Failure to remember: A common mistake is assuming SaaS means 'nothing to operate.' Fabric removes server patching and cluster provisioning work, but your team still owns bad joins, duplicate loads, over-broad access, runaway scans, broken dependencies and untested deployments. Another mistake is thinking a green pipeline run proves business correctness.

Say it back: You can point at every box in a small cloud data-flow diagram and say what it stores, what executes there, who authorizes it, what it costs broadly, and what proof would show that it worked.

DE15-02 | Identity, IAM and least privilege

Core: Access decision = principal + role + scope. Keep identities separate, privileges narrow and production execution independent of personal accounts.

Failure to remember: Two beginner failures recur. First, giving a human account broad Owner/Admin access because a lab is blocked. Second, confusing Microsoft Entra directory roles, Azure RBAC roles and Fabric workspace roles. They interact with different resource planes. Fix the missing permission at the layer that actually denies the action.

Say it back: You can explain why 'Contributor everywhere' is dangerous, distinguish workspace roles from Azure resource roles, and reason about effective permissions without guessing.

DE15-03 | Azure Storage concepts and OneLake

Core: OneLake is Fabric's logical lake; lakehouse Files stores arbitrary files and Tables exposes managed table data. Treat paths and environment bindings as explicit interfaces.

Failure to remember: Do not assume anything placed under Files automatically appears as a SQL-queryable table. Conversely, do not treat the Tables area as a dumping ground for random CSVs. Another mistake is hard-coding workspace/item display names into production code without a configuration strategy for DEV/TEST/PROD.

Say it back: You can decide whether a dataset belongs in Files or Tables, read a OneLake path without panic, and explain how OneLake differs from but interoperates with Azure storage concepts.

DE15-04 | Fabric workspace and item model

Core: Workspace = lifecycle/security/collaboration boundary; item = deployable Fabric artifact. Inventory dependencies before you deploy.

Failure to remember: A common anti-pattern is storing experimental, production and personal artifacts in one workspace because it is convenient. Another is assuming a workspace role automatically grants every data-plane permission in every connected external system.

Say it back: You can inspect a Fabric workspace and explain what each item does, what it depends on and why workspace placement matters for security and deployment.

DE15-05 | OneLake paths, files, tables and shortcuts

Core: Shortcut = reference, not independent copy. Decide from ownership, freshness, retention, security and availability requirements.

Failure to remember: Do not promise independent disaster recovery from a shortcut to the same upstream data. Do not assume a shortcut changes the underlying file format into Delta. And do not forget that consumers still need permissions appropriate to the path through which they access the data.

Say it back: You can explain when a shortcut is elegant and when a physical copy is justified, including the security and availability trade-off.

DE15-06 | Fabric Lakehouse and Delta tables

Core: Fabric Lakehouse = OneLake files + Delta tables + Spark engineering + an automatically provisioned read-oriented SQL analytics endpoint.

Failure to remember: A frequent mistake is trying to INSERT/UPDATE lakehouse Delta tables through the SQL analytics endpoint as if it were a normal writable SQL database. Another is assuming an arbitrary external Delta location will automatically appear in the endpoint without supported registration/shortcut behavior.

Say it back: You can say exactly which engine writes the table, which surface queries it, and where the durable data lives. You do not call every Fabric SQL surface a warehouse.

DE15-07 | Fabric notebooks

Core: Production notebook = explicit parameters + deterministic execution + modular logic + validation + reconcilable output + no secret/hard-coded environment leakage.

Failure to remember: Classic notebook failures include running cells out of order, leaving stale variables in memory, hard-coding PROD paths, installing libraries ad hoc in one session, printing secrets, and mixing exploratory display code with irreversible writes.

Say it back: You can open a notebook and quickly locate parameters, input contract, transformation logic, output write, validation and failure/recovery evidence.

DE15-08 | Spark in Fabric

Core: Fabric Spark is managed, not versionless. Record runtime evidence, version environments and carry M12 distributed-processing discipline into the cloud.

Failure to remember: Do not pip-install random incompatible versions in a notebook just to force parity with a laptop. Do not assume the newest runtime is automatically default in every workspace. Do not diagnose every slow Spark job as a capacity problem without checking plan, data shape and shuffle behavior from M12.

Say it back: You can identify the active Fabric runtime and explain why environment/runtime pinning matters for reproducibility and deployment.

DE15-09 | Fabric Data Pipelines

Core: Pipeline orchestration is state management. Commit progress only after durable validated output; parameterize runs and make retry semantics explicit.

Failure to remember: Common failures: updating watermark immediately after source read, using one giant activity with hidden side effects, swallowing notebook failure and still running the commit step, or retrying a non-idempotent external side effect without a deduplication key.

Say it back: You can read a Fabric pipeline and identify parameter flow, durable state, failure branches and the exact step that commits incremental progress.

DE15-10 | SQL analytics endpoint and warehouse choice

Core: Lakehouse SQL endpoint = read-oriented T-SQL window over lakehouse Delta tables; Warehouse = SQL-first writable warehousing item. Choose by ownership/write contract.

Failure to remember: Do not teach 'SQL endpoint = warehouse.' Do not promise full T-SQL DML against lakehouse Delta tables through the endpoint. Also do not create copies into Warehouse by default when the lakehouse endpoint already satisfies read analytics requirements.

Say it back: You can choose lakehouse endpoint versus Warehouse based on write model, engine preference and serving requirements, not merely because both show a SQL editor.

DE15-11 | Medallion architecture in Fabric

Core: Medallion = progressive trust and replay boundaries, not three fashionable folder names. Preserve raw evidence; make Silver quality explicit; make Gold consumer contracts stable.

Failure to remember: Anti-patterns include transforming Bronze until it is no longer raw, letting reports query unstable Bronze directly, copying the same cleansed table into multiple Gold folders, and treating Gold as 'aggregated only.' A Gold table can be detailed if it is consumer-ready and contractually stable.

Say it back: You can define one-sentence grain, quality and allowed-consumer rules for each layer and explain how to replay Silver/Gold from preserved evidence.

DE15-12 | Incremental loads and MERGE in cloud

Core: Incremental correctness = reliable change boundary + durable landing + idempotent target apply + commit progress only after validation.

Failure to remember: Never set the watermark to source max before copied rows are durably saved and validated. Avoid assuming timestamps are unique/monotonic. A lookback window reduces some late-data risk but does not solve arbitrarily late corrections unless the source exposes a reliable change sequence/CDC contract.

Say it back: You can draw candidate versus committed progress, name the idempotency key, and explain what happens on failure before and after each durable step.

DE15-13 | Permissions, connections and secrets

Core: Code references identities/connections/secret names; secret values live outside source. Diagnose 401/403 by identity + permission + scope + network, not by leaking credentials.

Failure to remember: Never print secrets to confirm they are correct. Avoid storing them in notebook output, .env committed to Git, pipeline parameter defaults, screenshots or README files. Also understand that having permission to manage a Key Vault resource is not necessarily the same as permission to read secret values.

Say it back: You can point to where identity is established, where secret material is stored, how it is retrieved, and what permissions allow that retrieval.

DE15-14 | Monitoring, reliability and cost awareness

Core: Observe run correctness, Spark behavior and capacity pressure separately. Optimize from evidence and keep freshness/reconciliation beside performance/cost.

Failure to remember: Do not respond to every slowdown by scaling capacity. A Cartesian join or tiny-file explosion can waste any capacity. Conversely, do not micro-optimize code while the capacity is visibly throttled by unrelated workloads. Also do not use a single successful run as a reliability claim.

Say it back: You can distinguish job-level failure, Spark performance regression and shared-capacity pressure, and you know which Fabric monitoring surface to consult for each.

DE15-15 | Git, deployment and environment separation

Core: Git versions supported definitions; deployment promotes metadata; bindings/config map dependencies per environment; production data is not a source-control artifact.

Failure to remember: Do not commit notebook outputs/secrets. Do not assume every Fabric item supports identical Git/deployment behavior. Do not treat a successful metadata deployment as proof that connections and downstream data contracts are correct. And never let PROD notebooks silently reference DEV lakehouses because an ID was hard-coded.

Say it back: You can explain what Git tracks, what deployment moves, what remains environment-specific and how you would verify that PROD dependencies point to PROD resources.

DE15-16 | End-to-end Fabric data engineering case

Core: End-to-end DE is a chain of explicit contracts. The strongest project proves state, replay, reconciliation, security, observability and release boundaries - not just screenshots.

Failure to remember: A flashy portal screenshot is not portfolio proof if you cannot explain why each item exists. Avoid adding Eventstream, Warehouse, Dataflow, Databricks or AI components merely to increase tool count. Complexity must earn its place through a requirement.

Say it back: You can narrate the architecture from one source change to one consumer query, including identity, durable state, validation, failure recovery, monitoring and deployment.